

Homework 4.3

Benjamin Loos

2026-04-12

Table of contents

0.1	Homework 4.3	1
0.2	A: Busiest Airports Analysis	2
0.3	B: Monte Carlo Numerical Integration step	3
0.3.1	Analysis	4
0.3.2	My plan:	5
0.3.3	my prompt in Copilot	5
0.3.4	general prompt with Copilot	7
0.3.5	comparison	7
0.4	Reflection	7
1	Code Appendix	18

0.1 Homework 4.3

the following document contains the answers and information for HW 4.3 in STAT 184 of spring 2026.

first, ill import the necessary libraries for the tables and graphs.

Warning: package 'rvest' was built under R version 4.5.3

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Warning: package 'tidyverse' was built under R version 4.5.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v forcats   1.0.1      v stringr   1.6.0
v lubridate 1.9.4      v tibble    3.3.1
v purrr     1.2.1      v tidyr     1.3.2
v readr     2.1.6
```

```
-- Conflicts ----- tidyverse_conflicts() --
x readr::col_factor()      masks scales::col_factor()
x purrr::discard()         masks scales::discard()
x dplyr::filter()          masks stats::filter()
x readr::guess_encoding()  masks rvest::guess_encoding()
x dplyr::lag()              masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

0.2 A: Busiest Airports Analysis

I started by taking the url where the airport data was, and stored it into a variable, “page”, using `read_html`.

then I put the tables from the url into a variable called “tables”.

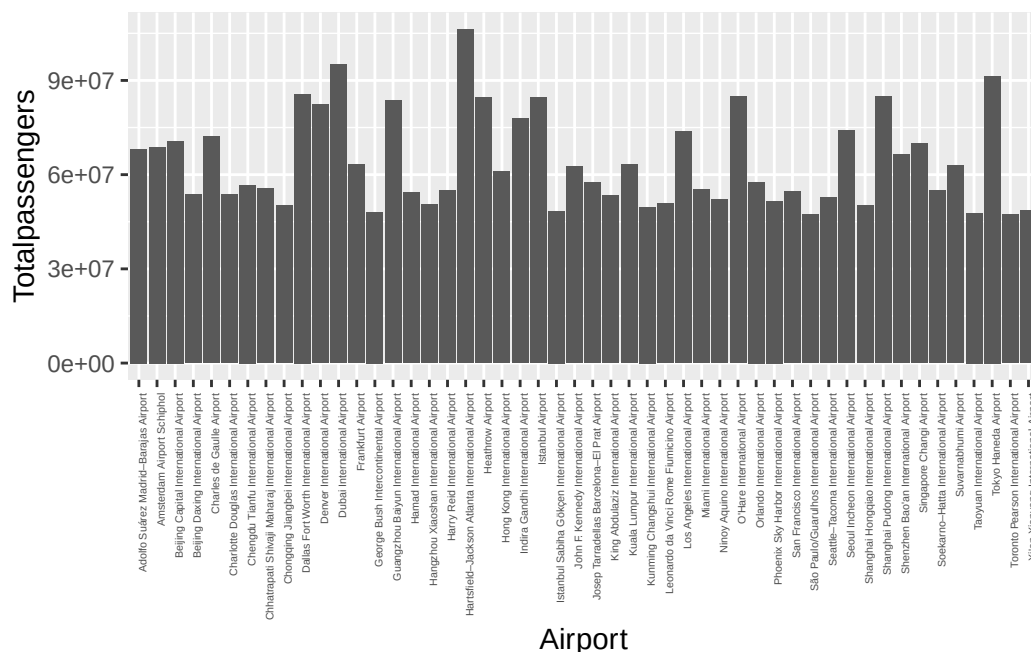
I will not be using the 2025 or 2026 dataset, so I will use tables to get the 2024 data.

The data needs to be clean up before using. It has unnecessary characters in TotalPassengers that need to be removed.

This is the clean 2024 ariport data table:

```
# A tibble: 50 x 8
  Rank Airport Location Country `Code(IATA/ICAO)` Totalpassengers Rankchange
  <dbl> <chr>      <chr>      <chr>      <chr>          <dbl>      <int>
1     1 Hartsfie~ Atlanta~ United~ ATL/KATL      106302208      NA
2     2 Dubai In~ Garhoud~ United~ DXB/OMDB      95200000      NA
3     3 Tokyo Ha~ Ōta, To~ Japan   HND/RJTT      91430000       1
4     4 Dallas F~ Dallas~~ United~ DFW/KDFW      85660127       1
5     5 Shanghai~ Pudong,~ China   PVG/ZSPD      84994548       5
6     6 O'Hare I~ Chicago~ United~ ORD/KORD      84851825       2
7     7 Heathrow~ Hilling~ United~ LHR/EGLL      84463000       1
8     8 Istanbul~ Arnavut~ Turkey  IST/LTFM      84457421      NA
9     9 Guangzho~ Baiyun~~ China   CAN/ZGGG      83582952       4
10    10 Denver I~ Denver,~ United~ DEN/KDEN      82427962       3
# i 40 more rows
# i 1 more variable: `change` <chr>
```

Using ggplot, I turned created a bar-plot comparing the total passengers each airport has.



0.2.0.1 analysis

The table of airport passenger totals gives a clear numeric overview of how traffic is distributed across many of the world's busiest airports, but the bar chart helps reveal the broader pattern in a way the table alone cannot. The chart makes it immediately visible that while a few airports operate at extremely high passenger volumes, most fall into a wide middle range, with substantial variation from airport to airport. Seeing the table and plot together highlights this spread.

0.3 B: Monte Carlo Numerical Integration step

My first step was to create the function as described by homework. The function is a chi-square density.

Then I created the bounds for the sampling. "a" is the min, "b" is the max, and "M" is the slope.

Now for the monte carlo intergration simulation function.

I created a vector for the different sample sizes.

Then simulated the data based on the sample sizes:

I then used the simulated data to create the estimates of the area under the curve.

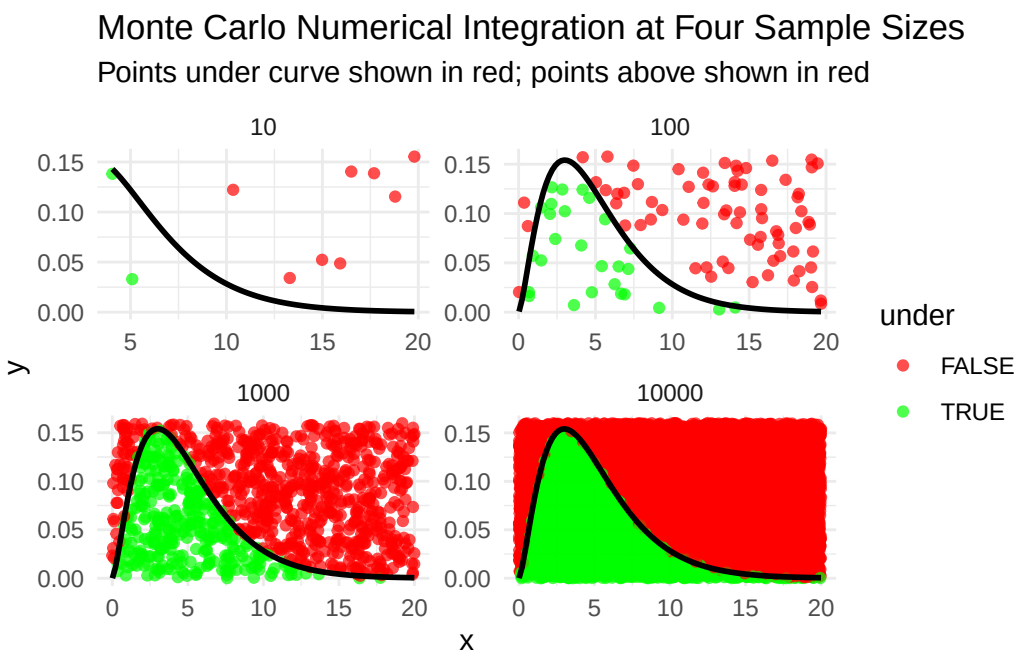
this is the table of estimates:

```
# A tibble: 4 x 2
      n estimate
  <dbl>   <dbl>
1    10    0.64
2   100    0.864
3  1000    1.01
4 10000    0.994
```

Also generated documentation for my monte carlo function:

Created file named 'mc_integrate.txt'.
 Edit the file and move it to the appropriate directory.

finally, used ggplot to create 4 graphs of the simulation data:



0.3.1 Analysis

The small-multiple plot makes it clear how increasing the number of Monte Carlo sample points improves the stability and accuracy of the numerical integration of the chi-square density function. In the panels using small sample sizes ($n = 10$ and $n = 100$), the points appear scattered unevenly, and the estimated integral varies widely, showing that randomness dominates when the number of samples is low. As the sample size increases to $n = 1000$ and especially $n = 10,000$, the green region under the curve becomes denser and more uniform, and the estimated area converges toward the true integral value.

##C GenAI

0.3.2 My plan:

step 1: load the dataset.
step 2: rename the columns into something meaningful.
step 3: pivot from wide to long.
step 4: drop missing measurements.
step 5: get the mean of the women's calcium.
step 6: plot the mean trend.

0.3.3 my prompt in Copilot

“step 1: load the dataset.”

New names:

```
* `y0` -> `y0...1`  
* `y1` -> `y1...2`  
* `y2` -> `y2...3`  
* `y3` -> `y3...4`  
* `y0` -> `y0...5`  
* `y1` -> `y1...6`  
* `y2` -> `y2...7`  
* `y3` -> `y3...8`
```

Rows: 16

Columns: 8

```
$ y0...1 <dbl> 87.3, 59.0, 76.7, 70.6, 54.9, 78.2, 73.7, 61.8, 85.3, 82.3, 68.~  
$ y1...2 <dbl> 86.9, 60.2, 76.5, 76.1, 55.1, 75.3, 70.8, 68.7, 84.4, 86.9, 65.~  
$ y2...3 <dbl> 86.7, 60.0, 75.7, 72.1, 57.2, 69.1, 71.8, 68.2, 79.2, 79.4, 72.~  
$ y3...4 <dbl> 75.5, 53.6, 69.5, 65.3, 49.0, 67.6, 74.6, 57.4, 67.0, 77.4, 60.~  
$ y0...5 <dbl> 83.8, 65.3, 81.2, 75.4, 55.3, 70.3, 76.5, 66.0, 76.7, 77.2, 67.~  
$ y1...6 <dbl> 85.5, 66.9, 79.5, 76.7, 58.3, 72.3, 79.9, 70.9, 79.0, 74.0, 70.~  
$ y2...7 <dbl> 86.2, 67.0, 84.5, 74.3, 59.1, 70.6, 80.4, 70.3, 76.9, 77.8, 68.~  
$ y3...8 <dbl> 81.2, 60.6, 75.2, 66.7, 54.2, 68.6, 71.6, 64.1, 70.3, 67.9, 65.~
```

“step 2: rename the columns into something meaningful.”

```
[1] "men_y0"    "men_y1"    "men_y2"    "men_y3"    "women_y0"  "women_y1"  "women_y2"  
[8] "women_y3"
```

“step 3: pivot from wide to long.”

```

Rows: 128
Columns: 4
$ id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3,~
$ sex     <fct> men, men, men, men, women, women, women, women, men, men, men,~
$ time    <int> 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 0,~
$ calcium <dbl> 87.3, 86.9, 86.7, 75.5, 83.8, 85.5, 86.2, 81.2, 59.0, 60.2, 60.~

```

“step 4: drop missing measurements.”

```

# A tibble: 1 x 1
  missing
  <int>
1      4

```

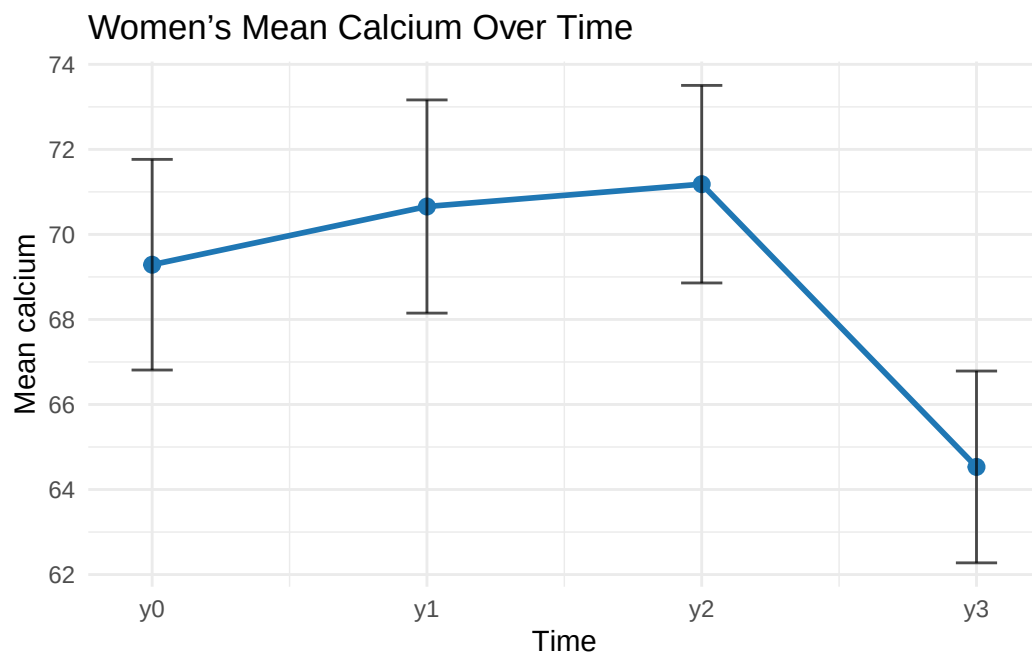
“step 5: get the mean of the women’s calcium.”

```

# A tibble: 4 x 5
  time      n mean_calcium    sd    se
  <int> <int>      <dbl> <dbl> <dbl>
1     0    16         69.3  9.91  2.48
2     1    16         70.7 10.0   2.51
3     2    16         71.2  9.29  2.32
4     3    16         64.5  9.02  2.26

```

“step 6: plot the mean trend.”



0.3.4 general prompt with Copilot

Prompt: “make a plot/graph with the data”

Generated:

0.3.5 comparison

The biggest difference I see is that the general prompt is written entirely in python. It also uses forloops a lot.I honeslty can't see any other differences because i can't understand the python code.

0.4 Reflection

In this course, I have learned a lot. I've learned the basic function in `r`, like `mutate`, `sum`, `pivot_long`. I've learned how to scrap data from websites, like the armed forces branch and rank. I've learned how to tidy and wrangle data into something usable. I've also learned what makes graphs and plots more readable/less-readable. I've learned that data scientists have to be responsible for the results they produce from their data.

{ { < pagebreak > } }

Copilot Apendix

```
library(rvest)
library(ggplot2)
library(dplyr)
library(scales)
library(tidyverse)
library(tidyr)

url<- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
page <- read_html(url)
tables <- page %>%
  html_nodes(".wikitable")
airport_data <- tables %>%
  .[[1]] %>% # extract table 1
  html_table(fill = TRUE)
airport_data$Totalpassengers <- airport_data$Totalpassengers %>%
  str_replace_all(",", "") %>%
  gsub("\\\\[.*?\\\\]", "", .) %>%
  as.numeric()

#| label: tbl-airports
#| tbl-cap: "Airports in the dataset"

airport_data

#| label: fig-airportPlot
```

```

#| fig-cap: "Comparison of the number of passengers between the most popular airports"
#| fig-pos: H
#| fig-height: 5
#| fig-alt: "Bar plot showing amount of passengers"
#| lst-label: lst-figure1
#| lst-cap: "Code Chunk for Making a airport bar plot"

ggplot(airport_data, aes(x = Airport, y = Totalpassengers)) +
  geom_bar(stat = "identity") +
  theme(axis.text.x = element_text(angle = 90, size = 4, hjust = 1))
f <- function(x) {
  dchisq(x, df = 5)
}
a <- 0
b <- 20 #bounds
M <- 0.16
mc_integrate <- function(n) {

  # random x in [a, b]
  x <- runif(n, min = a, max = b)

  # random y in [0, M]
  y <- runif(n, min = 0, max = M)

  # points under curve
  under_curve <- y < f(x)

  # area of bounding rectangle
  area_box <- (b - a) * M

  # Monte Carlo estimat
  estimate <- mean(under_curve) * area_box

  tibble(
    x = x,
    y = y,
    under = under_curve,
    n = n,
    estimate = estimate
  )
}
sizes <- c(10, 100, 1000, 10000)
sim_data <- map_df(sizes, mc_integrate)
estimates <- sim_data %>%
  group_by(n) %>%
  summarize(estimate = first(estimate))
estimates

```

```

prompt(mc_integrate, filename = "mc_integrate.txt")
ggplot(sim_data, aes(x = x, y = y, color = under)) +
  geom_point(size = 1.5, alpha = 0.7) +
  scale_color_manual(values = c("FALSE" = "red", "TRUE" = "green")) +
  geom_function(fun = f, color = "black", linewidth = 1) +
  facet_wrap(~ n, scales = "free", ncol = 2) +
  labs(
    title = "Monte Carlo Numerical Integration at Four Sample Sizes",
    subtitle = "Points under curve shown in red; points above shown in red",
    x = "x",
    y = "y"
  ) +
  theme_minimal()

library(tidyverse)

# Load data; repair duplicate column names automatically
calcium_wide <- readr::read_csv(
  "calcium.csv",
  show_col_types = FALSE,
  name_repair = "unique"
)

glimpse(calcium_wide)

stopifnot(ncol(calcium_wide) == 8)

names(calcium_wide) <- c(
  paste0("men_y", 0:3),
  paste0("women_y", 0:3)
)

names(calcium_wide)

calcium_long <- calcium_wide %>%
  mutate(id = row_number()) %>%
  pivot_longer(
    cols = -id,
    names_to = c("sex", "time"),
    names_pattern = "(men|women)_y(\\d+)",
    values_to = "calcium"
  ) %>%
  mutate(
    time = as.integer(time),

```

```

    sex = factor(sex, levels = c("men", "women"))
  )

glimpse(calcium_long)

calcium_clean <- calcium_long %>%
  drop_na(calcium)

# Quick check: how many did we drop?
calcium_long %>% summarise(missing = sum(is.na(calcium)))

women_means <- calcium_clean %>%
  filter(sex == "women") %>%
  group_by(time) %>%
  summarise(
    n = n(),
    mean_calcium = mean(calcium),
    sd = sd(calcium),
    se = sd / sqrt(n),
    .groups = "drop"
  )

women_means

p <- ggplot(women_means, aes(x = time, y = mean_calcium)) +
  geom_line(linewidth = 1, color = "#1f77b4") +
  geom_point(size = 2.5, color = "#1f77b4") +
  geom_errorbar(aes(ymin = mean_calcium - se, ymax = mean_calcium + se),
    width = 0.15, alpha = 0.7) +
  scale_x_continuous(breaks = 0:3, labels = paste0("y", 0:3)) +
  labs(
    title = "Women's Mean Calcium Over Time",
    x = "Time",
    y = "Mean calcium"
  ) +
  theme_minimal()

p

import numpy as np, matplotlib.pyplot as plt

# Prepare arrays
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), constrained_layout=True)

# --- Plot 1: Boxplots by time + group ---

```

```

ax = axes[0]
positions = []
box_data = []
labels = []
base = np.arange(len(time_order))
offset = 0.18
width = 0.28

for i, t in enumerate(time_order):
    a_vals = long[(long.group=='A') & (long.time==t)]['value'].dropna().values
    b_vals = long[(long.group=='B') & (long.time==t)]['value'].dropna().values
    box_data.extend([a_vals, b_vals])
    positions.extend([i - offset, i + offset])

bp = ax.boxplot(box_data, positions=positions, widths=width, patch_artist=True, showfliers=False)

# Color boxes alternating A/B
colors = ['#4C78A8', '#F58518'] * len(time_order)
for patch, c in zip(bp['boxes'], colors):
    patch.set_facecolor(c)
    patch.set_alpha(0.6)

# median lines
for med in bp['medians']:
    med.set_color('black')

# x-axis ticks at base positions
ax.set_xticks(base)
ax.set_xticklabels(time_order)
ax.set_xlabel('Time point')
ax.set_ylabel('Value')
ax.set_title('Distribution by time point (Groups A vs B)')

# Legend proxies
from matplotlib.patches import Patch
ax.legend(handles=[Patch(facecolor='#4C78A8', alpha=0.6, label='Group A'),
                  Patch(facecolor='#F58518', alpha=0.6, label='Group B')],
          frameon=False, loc='best')

# --- Plot 2: Means ± SEM ---
ax2 = axes[1]
for grp, color in [('A', '#4C78A8'), ('B', '#F58518')]:
    sub = summary[summary.group==grp].set_index('time').loc[time_order]
    x = np.arange(len(time_order))
    ax2.errorbar(x, sub['mean'].values, yerr=sub['sem'].values, marker='o', capsize=4,
                 color=color, label=f'Group {grp}')

```

```

ax2.set_xticks(x)
ax2.set_xticklabels(time_order)
ax2.set_xlabel('Time point')
ax2.set_ylabel('Mean value ( $\pm$  SEM)')
ax2.set_title('Mean trend over time')
ax2.legend(frameon=False, loc='best')

# Overall title
fig.suptitle('Calcium dataset overview', y=1.02, fontsize=12)

out_path = 'calcium_plots.png'
fig.savefig(out_path, dpi=200, bbox_inches='tight')
out_path

### Tool used
Microsoft Copilot (M365 Copilot / Copilot chat)

### Purpose of use
to generate a response specified by the homework

### What Copilot did not do
Copilot did not independently run code, access my private files, or submit work on my behalf.C

### Prompt / interaction log
Prompt: "step 1: load the dataset.

step 2: rename the columns into something meaningful.

step 3: pivot from wide to long.

step 4: drop missing measurements.

step 5: get the mean of the women's calcium.

step 6: plot the mean trend."

Generated{
  library(tidyverse)

# Load data; repair duplicate column names automatically
calcium_wide <- readr::read_csv(
  "calcium.csv",
  show_col_types = FALSE,
  name_repair = "unique"
)

```

```

glimpse(calcium_wide)

stopifnot(ncol(calcium_wide) == 8)

names(calcium_wide) <- c(
  paste0("men_y", 0:3),
  paste0("women_y", 0:3)
)

names(calcium_wide)

calcium_long <- calcium_wide %>%
  mutate(id = row_number()) %>%
  pivot_longer(
    cols = -id,
    names_to = c("sex", "time"),
    names_pattern = "(men|women)_y(\\d+)",
    values_to = "calcium"
  ) %>%
  mutate(
    time = as.integer(time),
    sex = factor(sex, levels = c("men", "women"))
  )

glimpse(calcium_long)

calcium_clean <- calcium_long %>%
  drop_na(calcium)

# Quick check: how many did we drop?
calcium_long %>% summarise(missing = sum(is.na(calcium)))

women_means <- calcium_clean %>%
  filter(sex == "women") %>%
  group_by(time) %>%
  summarise(
    n = n(),
    mean_calcium = mean(calcium),
    sd = sd(calcium),
    se = sd / sqrt(n),
    .groups = "drop"
  )

women_means

p <- ggplot(women_means, aes(x = time, y = mean_calcium)) +

```

```

geom_line(linewidth = 1, color = "#1f77b4") +
geom_point(size = 2.5, color = "#1f77b4") +
geom_errorbar(aes(ymin = mean_calcium - se, ymax = mean_calcium + se),
              width = 0.15, alpha = 0.7) +
scale_x_continuous(breaks = 0:3, labels = paste0("y", 0:3)) +
labs(
  title = "Women's Mean Calcium Over Time",
  x = "Time",
  y = "Mean calcium"
) +
theme_minimal()

p
}

```

Prompt: "make a plot/graph with the data"

Generated{

```
import numpy as np, matplotlib.pyplot as plt
```

```
# Prepare arrays
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), constrained_layout=True)
```

```
# --- Plot 1: Boxplots by time + group ---
```

```
ax = axes[0]
```

```
positions = []
```

```
box_data = []
```

```
labels = []
```

```
base = np.arange(len(time_order))
```

```
offset = 0.18
```

```
width = 0.28
```

```
for i, t in enumerate(time_order):
```

```
    a_vals = long[(long.group=='A') & (long.time==t)]['value'].dropna().values
```

```
    b_vals = long[(long.group=='B') & (long.time==t)]['value'].dropna().values
```

```
    box_data.extend([a_vals, b_vals])
```

```
    positions.extend([i - offset, i + offset])
```

```
bp = ax.boxplot(box_data, positions=positions, widths=width, patch_artist=True, showfliers=False)
```

```
# Color boxes alternating A/B
```

```
colors = ['#4C78A8', '#F58518'] * len(time_order)
```

```
for patch, c in zip(bp['boxes'], colors):
```

```
    patch.set_facecolor(c)
```

```
    patch.set_alpha(0.6)
```

```

# median lines
for med in bp['medians']:
    med.set_color('black')

# x-axis ticks at base positions
ax.set_xticks(base)
ax.set_xticklabels(time_order)
ax.set_xlabel('Time point')
ax.set_ylabel('Value')
ax.set_title('Distribution by time point (Groups A vs B)')

# Legend proxies
from matplotlib.patches import Patch
ax.legend(handles=[Patch(facecolor='#4C78A8', alpha=0.6, label='Group A'),
                   Patch(facecolor='#F58518', alpha=0.6, label='Group B')],
          frameon=False, loc='best')

# --- Plot 2: Means ± SEM ---
ax2 = axes[1]
for grp, color in [('A', '#4C78A8'), ('B', '#F58518')]:
    sub = summary[summary.group==grp].set_index('time').loc[time_order]
    x = np.arange(len(time_order))
    ax2.errorbar(x, sub['mean'].values, yerr=sub['sem'].values, marker='o', capsize=4,
                 color=color, label=f'Group {grp}')

ax2.set_xticks(x)
ax2.set_xticklabels(time_order)
ax2.set_xlabel('Time point')
ax2.set_ylabel('Mean value (± SEM)')
ax2.set_title('Mean trend over time')
ax2.legend(frameon=False, loc='best')

# Overall title
fig.suptitle('Calcium dataset overview', y=1.02, fontsize=12)

out_path = 'calcium_plots.png'
fig.savefig(out_path, dpi=200, bbox_inches='tight')
out_path
}

### Dates used
April-13-2026

### Notes / limitations
Copilot suggestions may contain errors or assumptions.

```



```

# points under curve
under_curve <- y < f(x)

# area of bounding rectangle
area_box <- (b - a) * M

# Monte Carlo estimat
estimate <- mean(under_curve) * area_box

tibble(
  x = x,
  y = y,
  under = under_curve,
  n = n,
  estimate = estimate
)
}

# 4. Run the simulation for four sample sizes
sizes <- c(10, 100, 1000, 10000)

sim_data <- map_df(sizes, mc_integrate)

# 5. Extract estimates for your write-up
estimates <- sim_data %>%
  group_by(n) %>%
  summarize(estimate = first(estimate))

estimates

prompt(mc_integrate, filename = "mc_integrate.txt")

ggplot(sim_data, aes(x = x, y = y, color = under)) +
  geom_point(size = 1.5, alpha = 0.7) +
  scale_color_manual(values = c("FALSE" = "red", "TRUE" = "green")) +
  geom_function(fun = f, color = "black", linewidth = 1) +
  facet_wrap(~ n, scales = "free", ncol = 2) +
  labs(
    title = "Monte Carlo Numerical Integration at Four Sample Sizes",
    subtitle = "Points under curve shown in blue; points above shown in gray",
    x = "x",
    y = "y"
  ) +
  theme_minimal()

```

1 Code Appendix

```
library(rvest)
library(ggplot2)
library(dplyr)
library(scales)
library(tidyverse)
library(tidyr)
url<- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
page <- read_html(url)
tables <- page %>%
  html_nodes(".wikitable")
airport_data <- tables %>%
  .[[1]] %>%      # extract table 1
  html_table(fill = TRUE)
airport_data$Totalpassengers <- airport_data$Totalpassengers %>%
  str_replace_all(",", "") %>%
  gsub("\\[.*?\\]", "", .) %>%
  as.numeric()

#| label: tbl-airports
#| tbl-cap: "Airports in the dataset"

airport_data

#| label: fig-airportPlot
#| fig-cap: "Comparison of the number of passengers between the most popular airports"
#| fig-pos: H
#| fig-height: 5
#| fig-alt: "Bar plot showing amount of passengers"
#| lst-label: lst-figure1
#| lst-cap: "Code Chunk for Making a airport bar plot"

ggplot(airport_data, aes(x = Airport, y = Totalpassengers)) +
  geom_bar(stat = "identity") +
  theme(axis.text.x = element_text(angle = 90, size = 4, hjust = 1))
f <- function(x) {
  dchisq(x, df = 5)
}
a <- 0
b <- 20 #bounds
M <- 0.16
mc_integrate <- function(n) {

  # random x in [a, b]
  x <- runif(n, min = a, max = b)
```

```

# random y in [0, M]
y <- runif(n, min = 0, max = M)

# points under curve
under_curve <- y < f(x)

# area of bounding rectangle
area_box <- (b - a) * M

# Monte Carlo estimat
estimate <- mean(under_curve) * area_box

tibble(
  x = x,
  y = y,
  under = under_curve,
  n = n,
  estimate = estimate
)
}
sizes <- c(10, 100, 1000, 10000)
sim_data <- map_df(sizes, mc_integrate)
estimates <- sim_data %>%
  group_by(n) %>%
  summarize(estimate = first(estimate))
estimates
prompt(mc_integrate, filename = "mc_integrate.txt")
ggplot(sim_data, aes(x = x, y = y, color = under)) +
  geom_point(size = 1.5, alpha = 0.7) +
  scale_color_manual(values = c("FALSE" = "red", "TRUE" = "green")) +
  geom_function(fun = f, color = "black", linewidth = 1) +
  facet_wrap(~ n, scales = "free", ncol = 2) +
  labs(
    title = "Monte Carlo Numerical Integration at Four Sample Sizes",
    subtitle = "Points under curve shown in red; points above shown in red",
    x = "x",
    y = "y"
  ) +
  theme_minimal()

library(tidyverse)

# Load data; repair duplicate column names automatically
calcium_wide <- readr::read_csv(
  "calcium.csv",
  show_col_types = FALSE,

```

```

    name_repair = "unique"
  )

  glimpse(calcium_wide)

  stopifnot(ncol(calcium_wide) == 8)

  names(calcium_wide) <- c(
    paste0("men_y", 0:3),
    paste0("women_y", 0:3)
  )

  names(calcium_wide)

  calcium_long <- calcium_wide %>%
    mutate(id = row_number()) %>%
    pivot_longer(
      cols = -id,
      names_to = c("sex", "time"),
      names_pattern = "(men|women)_y(\\d+)",
      values_to = "calcium"
    ) %>%
    mutate(
      time = as.integer(time),
      sex = factor(sex, levels = c("men", "women"))
    )

  glimpse(calcium_long)

  calcium_clean <- calcium_long %>%
    drop_na(calcium)

  # Quick check: how many did we drop?
  calcium_long %>% summarise(missing = sum(is.na(calcium)))

  women_means <- calcium_clean %>%
    filter(sex == "women") %>%
    group_by(time) %>%
    summarise(
      n = n(),
      mean_calcium = mean(calcium),
      sd = sd(calcium),
      se = sd / sqrt(n),
      .groups = "drop"
    )

```

```

)

women_means

p <- ggplot(women_means, aes(x = time, y = mean_calcium)) +
  geom_line(linewidth = 1, color = "#1f77b4") +
  geom_point(size = 2.5, color = "#1f77b4") +
  geom_errorbar(aes(ymin = mean_calcium - se, ymax = mean_calcium + se),
                width = 0.15, alpha = 0.7) +
  scale_x_continuous(breaks = 0:3, labels = paste0("y", 0:3)) +
  labs(
    title = "Women's Mean Calcium Over Time",
    x = "Time",
    y = "Mean calcium"
  ) +
  theme_minimal()

p

import numpy as np, matplotlib.pyplot as plt

# Prepare arrays
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), constrained_layout=True)

# --- Plot 1: Boxplots by time + group ---
ax = axes[0]
positions = []
box_data = []
labels = []
base = np.arange(len(time_order))
offset = 0.18
width = 0.28

for i, t in enumerate(time_order):
    a_vals = long[(long.group=='A') & (long.time==t)]['value'].dropna().values
    b_vals = long[(long.group=='B') & (long.time==t)]['value'].dropna().values
    box_data.extend([a_vals, b_vals])
    positions.extend([i - offset, i + offset])

bp = ax.boxplot(box_data, positions=positions, widths=width, patch_artist=True, showfliers=False)

# Color boxes alternating A/B
colors = ['#4C78A8', '#F58518'] * len(time_order)
for patch, c in zip(bp['boxes'], colors):
    patch.set_facecolor(c)
    patch.set_alpha(0.6)

```

```

# median lines
for med in bp['medians']:
    med.set_color('black')

# x-axis ticks at base positions
ax.set_xticks(base)
ax.set_xticklabels(time_order)
ax.set_xlabel('Time point')
ax.set_ylabel('Value')
ax.set_title('Distribution by time point (Groups A vs B)')

# Legend proxies
from matplotlib.patches import Patch
ax.legend(handles=[Patch(facecolor='#4C78A8', alpha=0.6, label='Group A'),
                  Patch(facecolor='#F58518', alpha=0.6, label='Group B')],
          frameon=False, loc='best')

# --- Plot 2: Means ± SEM ---
ax2 = axes[1]
for grp, color in [('A', '#4C78A8'), ('B', '#F58518')]:
    sub = summary[summary.group==grp].set_index('time').loc[time_order]
    x = np.arange(len(time_order))
    ax2.errorbar(x, sub['mean'].values, yerr=sub['sem'].values, marker='o', capsize=4,
                 color=color, label=f'Group {grp}')

ax2.set_xticks(x)
ax2.set_xticklabels(time_order)
ax2.set_xlabel('Time point')
ax2.set_ylabel('Mean value (± SEM)')
ax2.set_title('Mean trend over time')
ax2.legend(frameon=False, loc='best')

# Overall title
fig.suptitle('Calcium dataset overview', y=1.02, fontsize=12)

out_path = 'calcium_plots.png'
fig.savefig(out_path, dpi=200, bbox_inches='tight')
out_path

### Tool used
Microsoft Copilot (M365 Copilot / Copilot chat)

### Purpose of use
to generate a reponse specified by the homework

```

What Copilot did **not** do
Copilot did not independently run code, access my private files, or submit work on my behalf.

Prompt / interaction log

Prompt: "step 1: load the dataset.

step 2: rename the columns into something meaningful.

step 3: pivot from wide to long.

step 4: drop missing measurements.

step 5: get the mean of the women's calcium.

step 6: plot the mean trend."

Generated{

```
library(tidyverse)
```

```
# Load data; repair duplicate column names automatically
```

```
calcium_wide <- readr::read_csv(
```

```
  "calcium.csv",
```

```
  show_col_types = FALSE,
```

```
  name_repair = "unique"
```

```
)
```

```
glimpse(calcium_wide)
```

```
stopifnot(ncol(calcium_wide) == 8)
```

```
names(calcium_wide) <- c(
```

```
  paste0("men_y", 0:3),
```

```
  paste0("women_y", 0:3)
```

```
)
```

```
names(calcium_wide)
```

```
calcium_long <- calcium_wide %>%
```

```
  mutate(id = row_number()) %>%
```

```
  pivot_longer(
```

```
    cols = -id,
```

```
    names_to = c("sex", "time"),
```

```
    names_pattern = "(men|women)_y(\\d+)",
```

```
    values_to = "calcium"
```

```
  ) %>%
```

```
  mutate(
```

```
    time = as.integer(time),
```

```

    sex = factor(sex, levels = c("men", "women"))
  )

glimpse(calcium_long)

calcium_clean <- calcium_long %>%
  drop_na(calcium)

# Quick check: how many did we drop?
calcium_long %>% summarise(missing = sum(is.na(calcium)))

women_means <- calcium_clean %>%
  filter(sex == "women") %>%
  group_by(time) %>%
  summarise(
    n = n(),
    mean_calcium = mean(calcium),
    sd = sd(calcium),
    se = sd / sqrt(n),
    .groups = "drop"
  )

women_means

p <- ggplot(women_means, aes(x = time, y = mean_calcium)) +
  geom_line(linewidth = 1, color = "#1f77b4") +
  geom_point(size = 2.5, color = "#1f77b4") +
  geom_errorbar(aes(ymin = mean_calcium - se, ymax = mean_calcium + se),
    width = 0.15, alpha = 0.7) +
  scale_x_continuous(breaks = 0:3, labels = paste0("y", 0:3)) +
  labs(
    title = "Women's Mean Calcium Over Time",
    x = "Time",
    y = "Mean calcium"
  ) +
  theme_minimal()

p
}

```

Prompt: "make a plot/graph with the data"

Generated{
import numpy as np, matplotlib.pyplot as plt

```

# Prepare arrays
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), constrained_layout=True)

# --- Plot 1: Boxplots by time + group ---
ax = axes[0]
positions = []
box_data = []
labels = []
base = np.arange(len(time_order))
offset = 0.18
width = 0.28

for i, t in enumerate(time_order):
    a_vals = long[(long.group=='A') & (long.time==t)]['value'].dropna().values
    b_vals = long[(long.group=='B') & (long.time==t)]['value'].dropna().values
    box_data.extend([a_vals, b_vals])
    positions.extend([i - offset, i + offset])

bp = ax.boxplot(box_data, positions=positions, widths=width, patch_artist=True, showfliers=False)

# Color boxes alternating A/B
colors = ['#4C78A8', '#F58518'] * len(time_order)
for patch, c in zip(bp['boxes'], colors):
    patch.set_facecolor(c)
    patch.set_alpha(0.6)

# median lines
for med in bp['medians']:
    med.set_color('black')

# x-axis ticks at base positions
ax.set_xticks(base)
ax.set_xticklabels(time_order)
ax.set_xlabel('Time point')
ax.set_ylabel('Value')
ax.set_title('Distribution by time point (Groups A vs B)')

# Legend proxies
from matplotlib.patches import Patch
ax.legend(handles=[Patch(facecolor='#4C78A8', alpha=0.6, label='Group A'),
                  Patch(facecolor='#F58518', alpha=0.6, label='Group B')],
          frameon=False, loc='best')

# --- Plot 2: Means ± SEM ---
ax2 = axes[1]
for grp, color in [('A', '#4C78A8'), ('B', '#F58518')]:
    sub = summary[summary.group==grp].set_index('time').loc[time_order]

```

```

x = np.arange(len(time_order))
ax2.errorbar(x, sub['mean'].values, yerr=sub['sem'].values, marker='o', capsize=4,
             color=color, label=f'Group {grp}')

ax2.set_xticks(x)
ax2.set_xticklabels(time_order)
ax2.set_xlabel('Time point')
ax2.set_ylabel('Mean value ( $\pm$  SEM)')
ax2.set_title('Mean trend over time')
ax2.legend(frameon=False, loc='best')

# Overall title
fig.suptitle('Calcium dataset overview', y=1.02, fontsize=12)

out_path = 'calcium_plots.png'
fig.savefig(out_path, dpi=200, bbox_inches='tight')
out_path
}

### Dates used
April-13-2026

### Notes / limitations
Copilot suggestions may contain errors or assumptions.

library(rvest)
library(ggplot2)
library(dplyr)
library(scales)
library(tidyverse)
library(tidyr)

#Part A code:
url<- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
page <- read_html(url)

tables <- page %>%
  html_nodes(".wikitable")

airport_data <- tables %>%
  .[[1]] %>% # extract table 1
  html_table(fill = TRUE)

```

```

airport_data$Totalpassengers <- airport_data$Totalpassengers %>%
  str_replace_all(",", "") %>%
  gsub("\\[..*?\\]", "", .) %>%
  as.numeric()

ggplot(airport_data, aes(x = Airport, y = Totalpassengers)) +
  geom_bar(stat = "identity") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))

#Part B code:

f <- function(x) {
  dchisq(x, df = 5)
}
a <- 0
b <- 20 #bounds
M <- 0.16
# the Monte Carlo simulation
mc_integrate <- function(n) {

  # random x in [a, b]
  x <- runif(n, min = a, max = b)

  # random y in [0, M]
  y <- runif(n, min = 0, max = M)

  # points under curve
  under_curve <- y < f(x)

  # area of bounding rectangle
  area_box <- (b - a) * M

  # Monte Carlo estimat
  estimate <- mean(under_curve) * area_box

  tibble(
    x = x,
    y = y,
    under = under_curve,
    n = n,
    estimate = estimate
  )
}

# 4. Run the simulation for four sample sizes
sizes <- c(10, 100, 1000, 10000)

```

```

sim_data <- map_df(sizes, mc_integrate)

# 5. Extract estimates for your write-up
estimates <- sim_data %>%
  group_by(n) %>%
  summarize(estimate = first(estimate))

estimates

prompt(mc_integrate, filename = "mc_integrate.txt")

ggplot(sim_data, aes(x = x, y = y, color = under)) +
  geom_point(size = 1.5, alpha = 0.7) +
  scale_color_manual(values = c("FALSE" = "red", "TRUE" = "green")) +
  geom_function(fun = f, color = "black", linewidth = 1) +
  facet_wrap(~ n, scales = "free", ncol = 2) +
  labs(
    title = "Monte Carlo Numerical Integration at Four Sample Sizes",
    subtitle = "Points under curve shown in blue; points above shown in gray",
    x = "x",
    y = "y"
  ) +
  theme_minimal()

```